6.2.1. Giới thiệu về MongoDB

MongoDB là một chương trình cơ sở dữ liệu mã nguồn mở được thiết kế theo kiểu hướng đối tượng trong đó các bảng được cấu trúc một cách linh hoạt cho phép các dữ liệu lưu trên bảng không cần phải tuân theo một dạng cấu trúc nhất định nào. Chính do cấu trúc linh hoạt này nên MongoDB có thể được dùng để lưu trữ các dữ liệu có cấu trúc phức tạp và đa dạng và không cố định (hay còn gọi là Big Data).

Lợi thế của MongoDB so với các cơ sở dữ liệu dạng quan hệ(RDBMS):

- Ít Schema hơn: MongoDB là một cơ sở dữ liệu dựa trên Document, trong đó một Collection giữ các Document khác nhau. Số trường, nội dung và kích cỡ của Document này có thể khác với Document khác.

- Cấu trúc của một đối tượng là rõ ràng.

- Không có các Join phức tạp.

- Khả năng truy vấn sâu hơn. MongoDB hỗ trợ các truy vấn động trên các Document bởi sử dụng một ngôn ngữ truy vấn dựa trên Document mà mạnh mẽ như SQL.

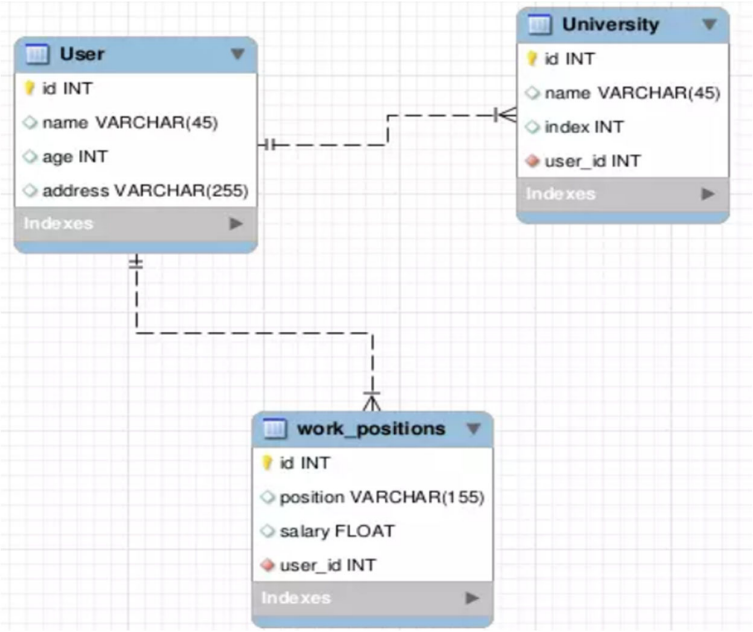
- MongoDB dễ dàng để mở rộng.

- Việc chuyển đổi/ánh xạ của các đối tượng ứng dụng đến các đối tượng cơ sở dữ liệu là không cần thiết.

- Sử dụng bộ nhớ nội tại để lưu giữ phần công việc, giúp truy cập dữ liệu nhanh hơn.

So sánh MongoDB với cơ sở dữ liệu quan hệ:

- Thiết kế database trong Mysql:



- Trong khi trong thiết kế **Schema** của **MongoDB** sẽ chỉ có một **Collection**:

```
{
  _id: user_id,
  name: user_name,
  email: user_email,
  age: age,
  address: user_address,
  universities: [
    {name: university_name, index: university_index}
  ],
  work_positions: [
    {position: work_position, salary: user_salary}
  ]
}
```

Các phương thức cơ bản trong MongoDB so với cơ sở dữ liệu quan hệ:

Operation	Mysql	MongoDB
Create	Insert	Insert
Read	Select	Find
Update	Update	Update
Delete	Delete	Remove
Table	Table	Collection
Row	Row	Document

Lưu ý khi sử dụng Mongodb:

- MongoDB sinh ra để tăng tốc độ truy xuất dữ liệu, phù hợp cho các ứng dụng cần tốc độ phản hồi nhanh (realtime như facebook chẳng hạn). Còn các tác nghiệp cần tính toàn vẹn dữ liệu (trong banking) thì Nosql sẽ ko bao giờ là 1 giải pháp cả mà người ta sẽ dùng Mysql.

- MongoDB không có tính ràng buộc, một điều tồi tệ trong Database vì vậy sẽ rất cần sự cẩn thận khi thao tác trên các collection có quan hệ dữ liệu với nhau.

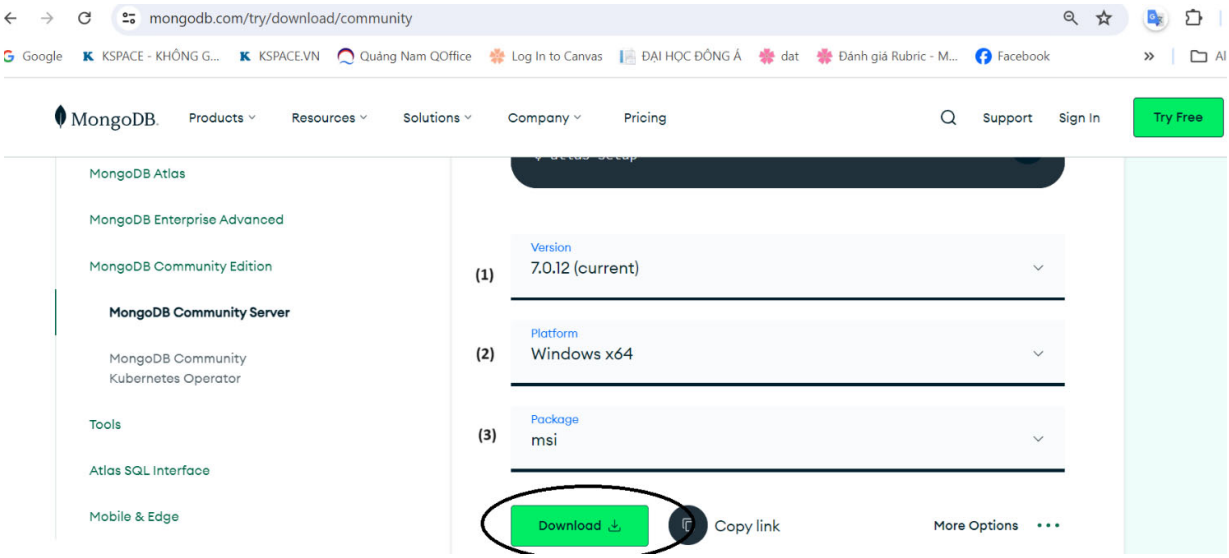
- MongoDB đẩy trách nghiệm thao tác Database cho tầng ứng dụng nên sẽ tốn tài nguyên(tài nguyên bây giờ không còn là vấn đề quá lớn nữa).

- MongoDb có thể mở trộng theo chiều ngang (scale out) phương pháp tăng cường khả năng lưu trữ và xử lý là dùng nhiều máy tính phân tán. MongoDb còn có thể mở rộng theo chiều dọc (scale up) tăng cấu hình server.

- MongoDb có thể dùng nhiều máy tính phân tán để lưu trữ dữ liệu nên chi phí sẽ rẻ hơn Mysql . Mysql sử dụng những máy chủ hàng khủng, độc quyền nên sẽ đắt đỏ hơn.

6.2.2. Cài đặt MongoDB trên Windows

+ **Bước 1:** Đầu tiên các bạn vào đường dẫn sau để download file cài đặt của MongoDB tại đây: **https://www.mongodb.com/try/download/community** . Chọn **Download**



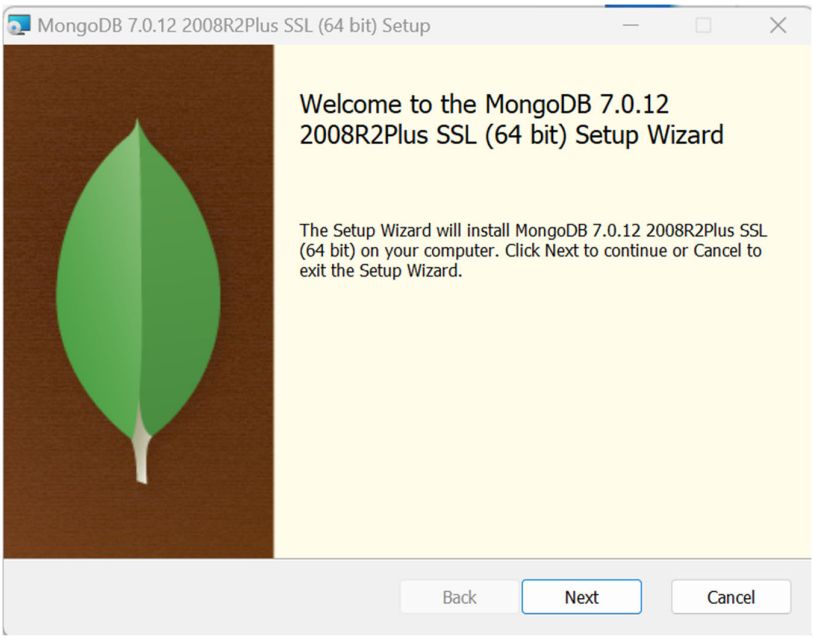
Thông tin lựa chọn:

(1) Là phiên bản hiện tại của Mongodb, phiên bản cộng đồng (Free).

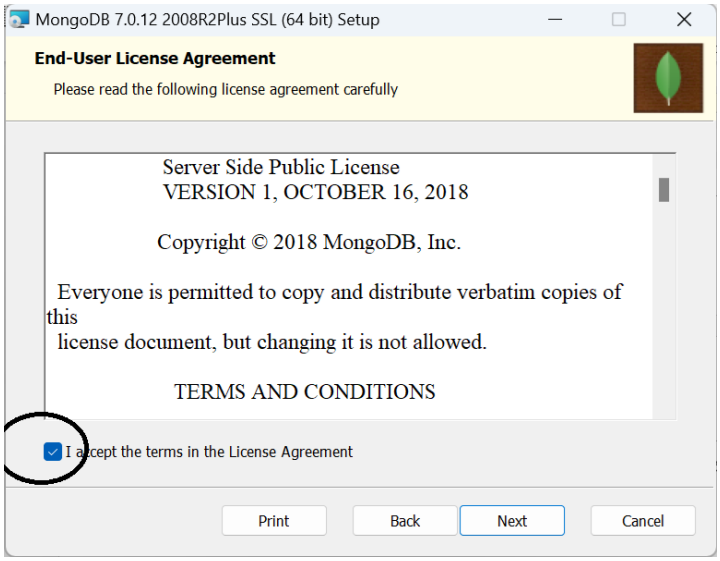
(2) Hệ điều hành (ở đây chúng ta chọn Windows).

(3) Loại file cài đặt (ở đây chúng ta chọn file *.msi).

+ **Bước 2:** Sau khi tải file cài đặt về, các bạn nháy đúp chuột vào file để bắt đầu quá trình cài đặt => và bấm **Next** để chuyển sang bước tiếp theo.



+ **Bước 3:** Ở bước này các bạn tích vào I accept the terms in the License Agreement => sau đó bấm vào Next để sang bước tiếp theo.

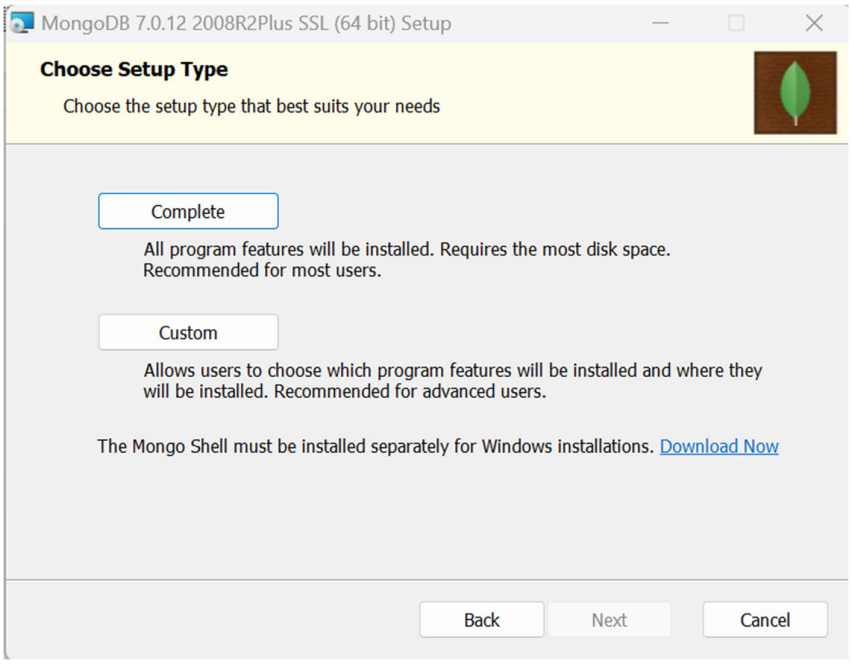


+ **Bước 4:** Ở đây sẽ có hai option đó là hoặc bạn bấm vào Complete hoặc Custom (Tùy chọn).

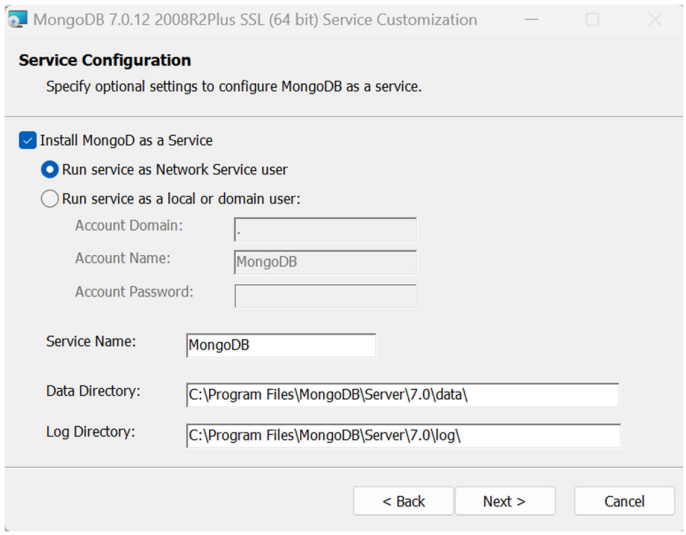
Nếu bạn chọn **Complete** thì tất cả các chức năng sẽ được cài đặt và tất nhiên là sẽ chiếm nhiều dung lượng ổ cứng hơn.

Còn nếu bạn chọn **Custom** thì bạn sẽ chỉ chọn những chức năng bạn muốn cài đặt và tùy thuộc vào chức năng bạn chọn dung lượng ổ cứng cài đặt sẽ giảm đi.

=> Chúng ta thấy **MongoDB** gợi ý chúng ta (người dùng mới) thì nên cài đặt theo **option** đầu tiên (**Complete**). Vì vậy các bạn cứ bấm vào **Complete** để tiếp tục nha.



+ **Bước 5:** Tiếp đến bước cấu hình service.



(1) Cài đặt MongoDB như là một service. Nó giống như những service bình thường của hệ điều hành. Bạn có thể bật/tắt được.

(2) Chạy service với vai trò là người sử dụng mạng dịch vụ.

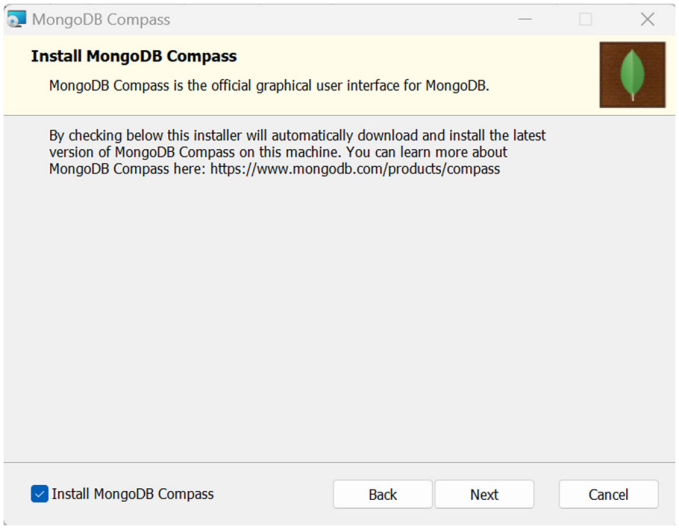
(3) Nơi lưu trữ Database.

(4) Nơi chứa file log (các server chạy đều có file log, các bạn có thể xem log file xem server đã chạy các service nào, gặp lỗi ở đâu…)

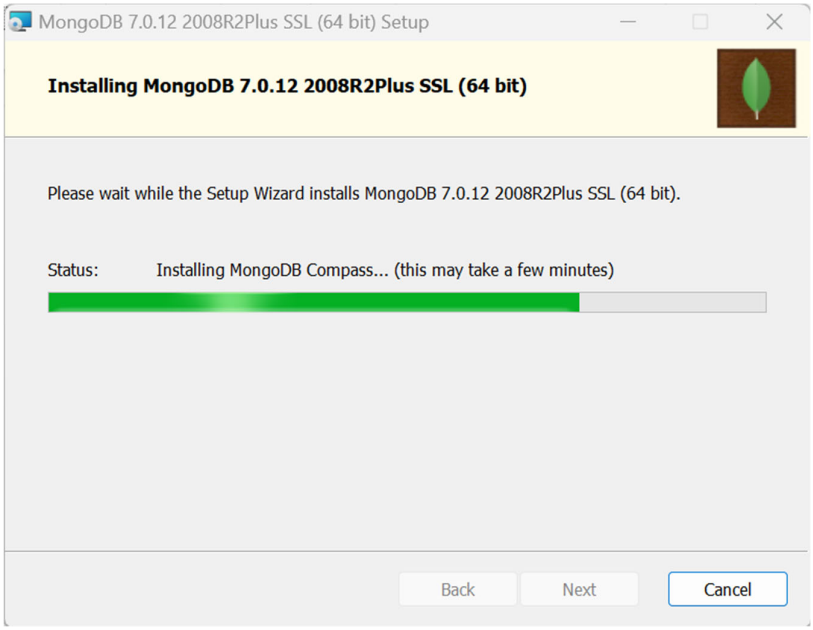
=> Sau đó các bạn bấm **Next** để tiếp tục.

+ **Bước 6:** Bước tiếp theo này thì các bạn sẽ lựa chọn là có cài **MongoDB Compass** hay không. Cho bạn nào chưa biết thì MongoDB Compass là một công cụ, chính xác hơn là một giao diện người dùng giúp cho việc thao tác với dữ liệu trong Mongodb trực quan hơn.

=> Bạn tích và ô **Install MongoDB Compass** => và bấm **Next** để đến bước cài đặt.

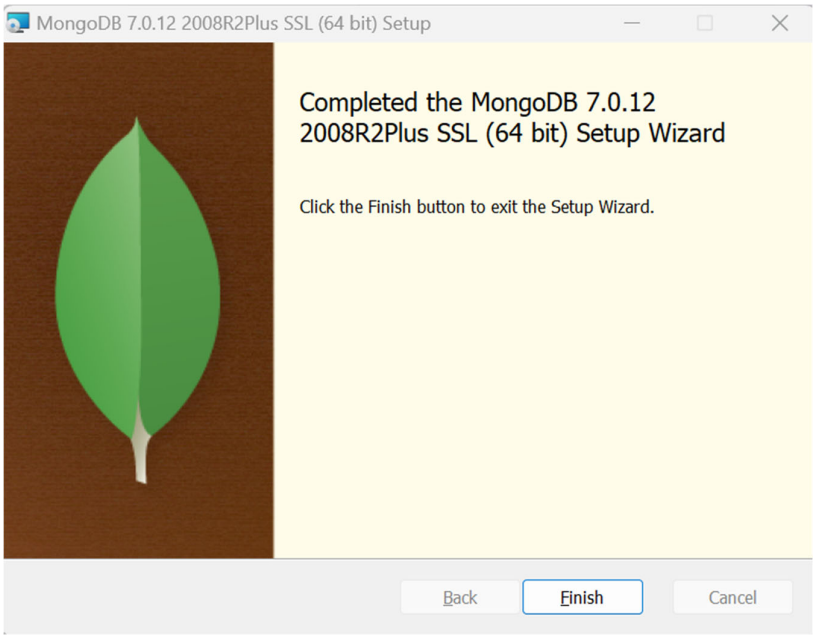


+ **Bước 7:** Đến bước này thì quá trình cấu hình đã xong và bắt đầu cài đặt. Các bạn bấm vào **Install** để bắt đầu quá trình cài đặt.

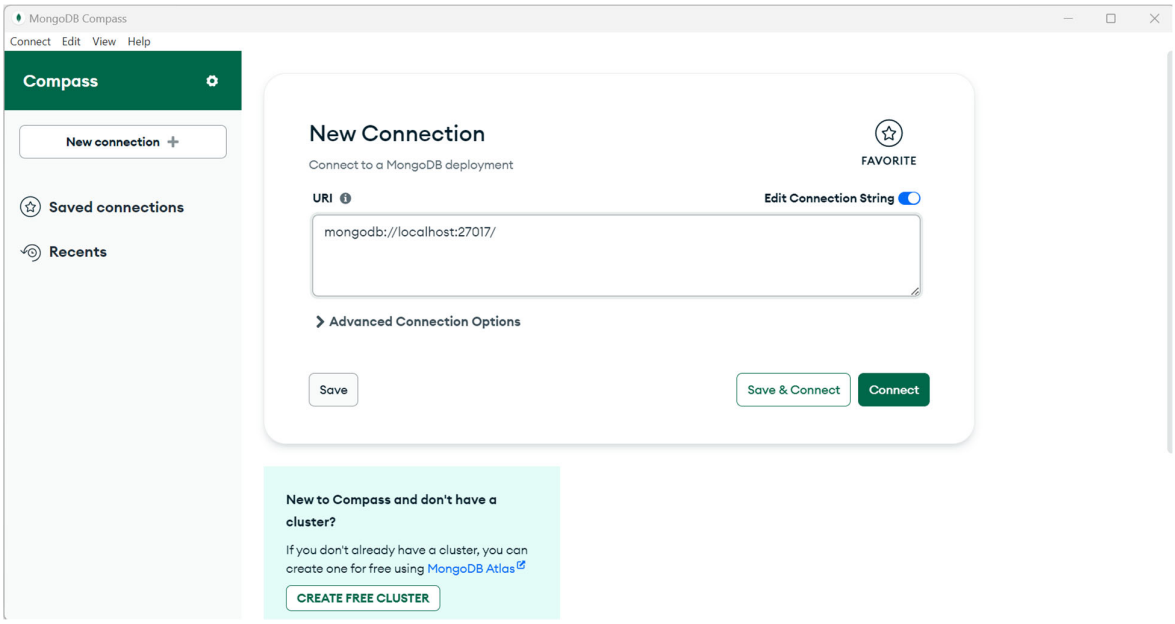


Quá trình cài đặt đang diễn ra, nhanh thì có thể 2-3 phút là xong, chậm có thể mất 5-7 phút tùy thuộc vào kết nối mạng và cấu hình máy nữa.

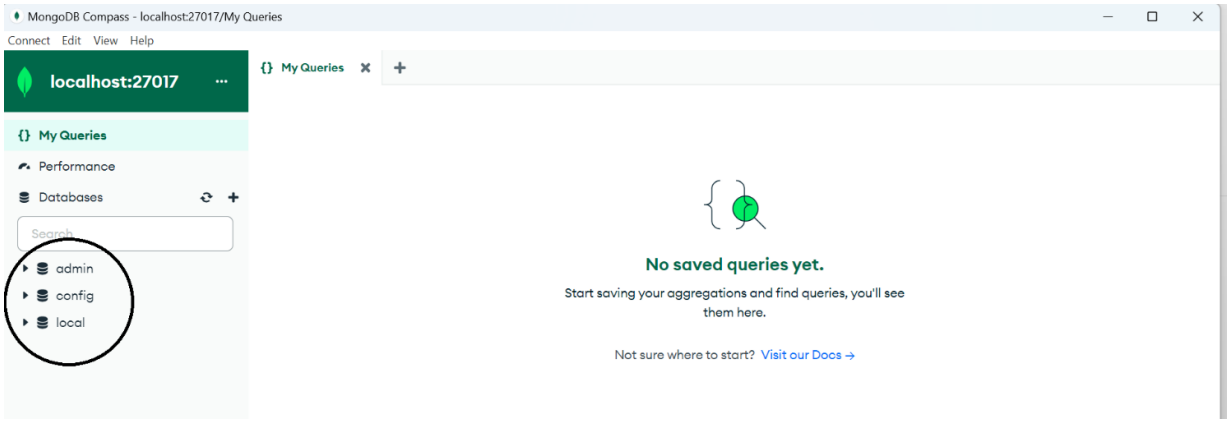
+ **Bước 8:** Sau khi cài đặt xong các bạn bấm vào **Finish** để hoàn tất quá trình cài đặt.



Sau khi cài đặt các bạn chọn **Start** để chạy **Mongodb Compass** để vào diện như bên dưới. Đây chính là giao diện chúng ta sẽ làm việc với **mongodb**.



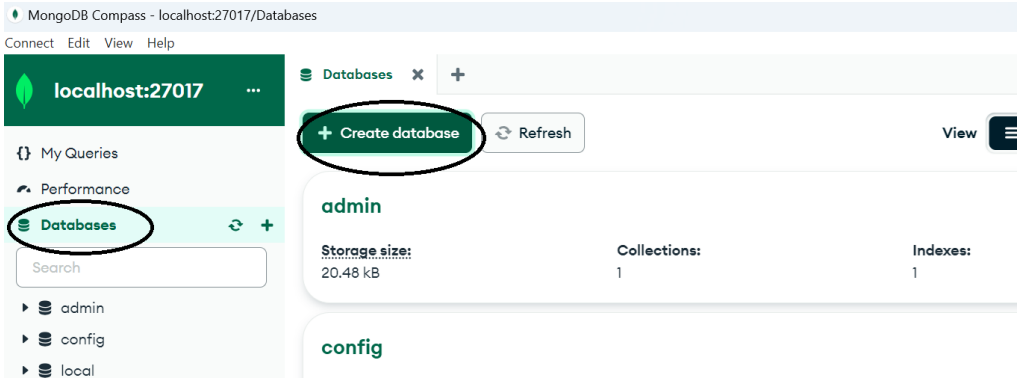
Các bạn chọn **Connect** để kết nối với **MongoDB** chạy trên localhost, với đường dẫn mặc định là: **mongodb://localhost:27017**



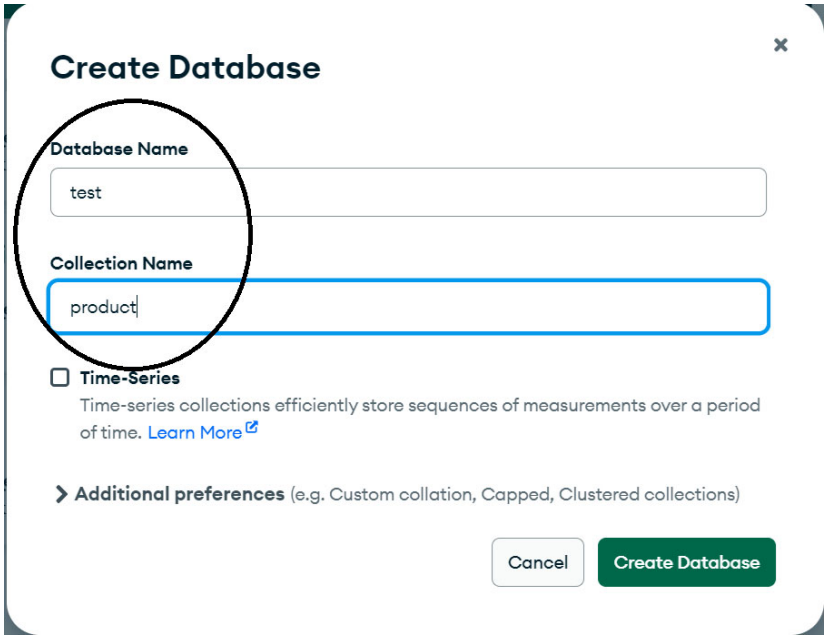
Mặc định **MongoDB** đã có sẵn 3 cơ sở dữ liệu là: **admin, config, local**.

6.2.3. Tạo cơ sở dữ liệu trong MongoDB

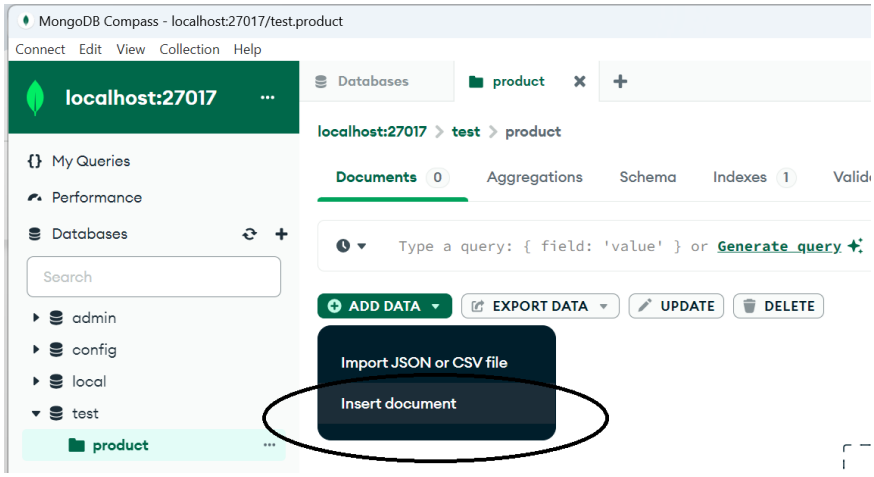
- Trên **MongoDB Compass**, chọn **Databases**, chọn **Create Database**



- Gõ tên cơ sở dữ liệu: **test**, gõ Bảng dữ liệu (Collection Name): **product**, chọn **Create Database**

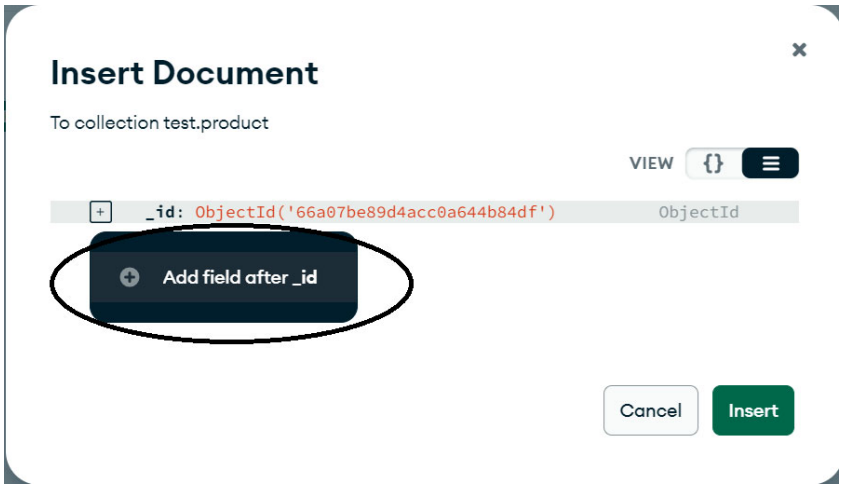


- Chọn **ADD DATA**, chọn **Insert document**



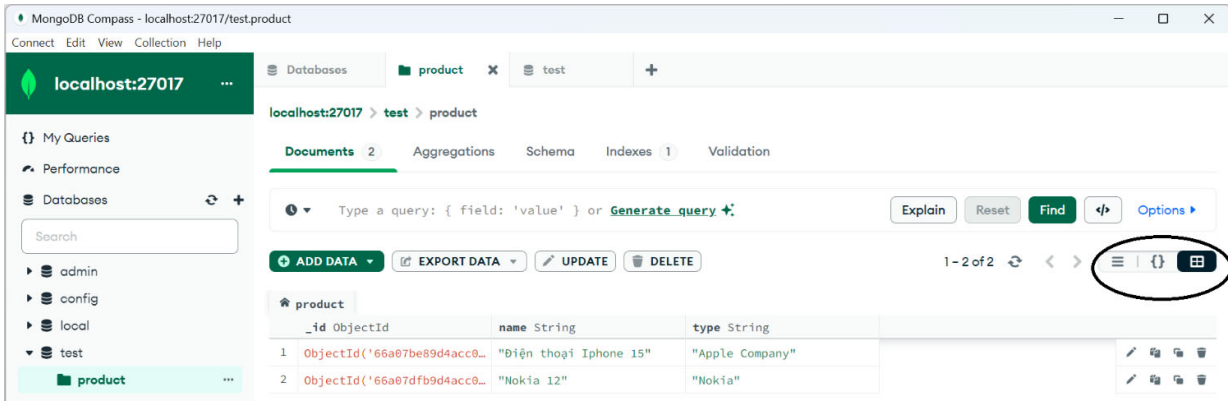
- Chọn chế độ nhập dữ liệu cho đơn giản hơn:

- Đã có trường mặc định _id, Thêm trường dữ liệu **Add fiels after _id**



- Ví dụ nhập dữ liệu như bên dưới, chọn **Insert**

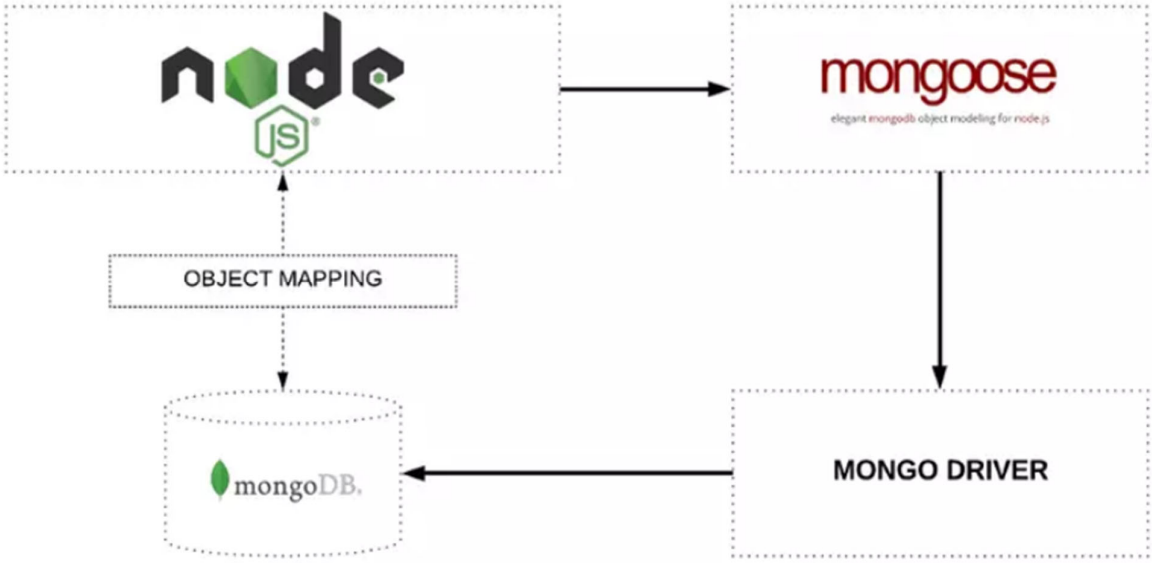
- Các bạn có thể thêm dòng dòng dữ liệu vào bảng **product**. Có thể xem chế độ hiển thị dữ liệu phía bên phải:



Lưu ý: Ngoài chương trình **MongoDB Compass**, chúng ta cũng có thể sử dụng nhiều phần mềm khác quản lý cơ sở dữ liệu MongoDB như: **Robo 3T**, …

6.2.4. Giới thiệu về Mongoose

Mongoose là một thư viện mô hình hóa đối tượng (**Object Data Model - ODM**) cho **MongoDB** và **Node.js**. Nó quản lý mối quan hệ giữa dữ liệu, cung cấp sự xác nhận giản đồ và được sử dụng để dịch giữa các đối tượng trong mã và biểu diễn các đối tượng trong MongoDB.



Hình 6.1: Mongoose

Để cài đặt **Mongoose** trong **Nodejs** các bạn dùng lệnh sau: **npm install mongoose**

Hoặc **npm i mongoose validation**: nếu muốn sử dụng các điều kiện

6.2.5. Kết nối cơ sở dữ liệu MongoDB trong Nodejs

Ví dụ 6.1: Viết ứng dụng kiểm tra kết nối cơ sở dữ liệu MongoDB đã tạo ở mục 6.2.3

- Tạo thư mục **Chuong6** ở đường dẫn: **D:\THNodejs\Chuong06**

- Tạo thư mục **Vidu6_1** trong thư mục **Chuong6**

- Mở chương trình **VSCode** lên và mở thư mục **Vidu6_1** trong **VSCode**

- Tạo tập tin **Vidu6_1.js** trong thư mục **Vidu6_1**

- Khởi tạo **npm** với dòng lệnh: **npm init**, bấm **Enter** vài lần rồi gõ **yes**

- Cài đặt **npm** với dòng lệnh: **npm install**

- Cài đặt **Mongoose** với dòng lệnh: **npm i mongoose**

- Tạo tập tin **Vidu6_1.js** và viết code như bên dưới:

```
EXPLORER
VIDU6_1
  node_modules
  package-lock.json
  package.json
  Vidu6_1.js
```

```js
//Tạo đối tượng mongose
let mongoose = require('mongoose');

//Chuỗi kết nối đến cơ sở dữ liệu MongoDB
const mongodb_url = 'mongodb://localhost:270173/test'

//Kiểm tra kết nối đến cơ sở dữ liệu MongoDB
mongoose.connect(mongodb_url, { useNewUrlParser: true })
    .then(() => {
        console.log("Database kết nối thành công");
    })
    .catch(err => {
        console.log("Database bị lỗi kết nối!");
    })
```

- Biên dịch chương trình sẽ cho kết quả:

```
PS D:\THNodejs\Chuong06\Vidu6_1> node Vidu6_1.js
(node:16244) [MONGODB DRIVER] Warning: useNewUrlParser is a deprecated o
rsion 4.0.0 and will be removed in the next major version
(Use `node --trace-warnings ...` to show where the warning was created)
Database kết nối thành công
```

- Nếu các bạn thử thay đổi cổng kết nối của **MongoDB**, ví dụ thay đổi cổng là: 270173, khi biên dịch chương trình sẽ thông báo lỗi kết nối:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\THNodejs\Chuong06\Vidu6_1> node Vidu6_1.js
Database bị lỗi kết nối!
PS D:\THNodejs\Chuong06\Vidu6_1>
```

6.2.6. SchemaTypes và Options SchemaTypes của mongoose

SchemaTypes giúp bạn định dạng kiểu dữ liệu mà bạn mong muốn. Mỗi khi có bất kì tác động nào thì Schema sẽ dựa vào SchemaTypes để kiểm tra kiểu dữ liệu đó có đúng với kiểu đã đặt trước đó không.

Chúng ta có một vài **SchemaTypes** trong **Mongoose** như sau:

- String: chuỗi ký tự
- Number: Số nguyên Int32, Int64
- Date: Ngày tháng
- Boolean: true / false
- ObjectId: Kiểu Id riêng của MongoDB (tự động generate ra khi tạo một document)
- Array: Mảng
- Decimal128: Số thực
- Object

Ví dụ dưới đây sẽ khởi tạo một **Schema**:

- Định dạng kiểu dữ liệu cho các **Document** (Filed): username, password

- **Collection** (Table) Account

- Cơ sở dữ liệu **Quanly**.

```
1   //Require gói mongoose
2   const mongoose = require('mongoose');
3
4   //Khai báo kết nối dữ liệu đến cơ sở dữ liệu Quanly
5   //Kết nối collection: Account
6   mongoose.connect('mongodb://localhost:27017/Quanly', {
7       useNewUrlParrser: true,
8       useUnifielTopology: true
9   });
10
11  //Tạo Schema của mongoose
12  const Schema = mongoose.Schema;
13
14  const accountschema = new Schema({
15      username: String,
16      password: String
17  }, {
18      collection: 'Account'
19  });
```

Như ví dụ trên chúng ta đã tạo được 1 schema là: **accountschema**

Schema có những **option** hỗ trợ việc kiểm tra và ràng buộc dữ liệu. **Options Type** (Dùng trong tất cả các SchemaTypes) thì ta có:

- required : (boolean) bắt buộc phải có dữ liệu
- default: (không sử dụng với required) (boolean) nếu không nhập dữ liệu thì sẽ lấy giá trị mặc định này
- select: chỉ định cách trường được trả về khi truy vấn
- validate: (function) kiểm tra gía trị của thuộc tính (bằng regex,..).
- get: (function) chỉ định các giá trị lấy vào của thuộc tính sử dụng Object.defineProperty()
- set: (function) chỉ định các giá trị được đặt vào của thuộc tính sử dụng Object.defineProperty()
- alias : tên gọi thay thế của trường này.

Ví dụ **Schema** với những **option** phù hợp:

```js
Schema.js > ...
const mongoose = require('mongoose');
const validator = require('validator');

const userSchema = mongoose.Schema(
    {
        name: {
            type: String,
            trim: true,
        },
        age: {
            type: Number,
            trim: true,
        },
        avatar: {
            type: Number,
            trim: true,
            default: '123456789',
        },
        lastLogin: {
            type: Date,
            default: Date.now,
        },
        username: {
            type: String,
            require: true,
            trim: true,
        },
        email: {
            type: String,
            require: true,
            unique: true,
            trim: true,
            lowercase: true,
            validator(value) {
                if (!validator.isEmail(value)) {
                    throw new Error('Lỗi Email');
                }
            },
        },
        password: {
            type: String,
            require: true,
            trim: true,
            minlength: 8,
            validate(value) {
                if (!value.match(/\d/) || value.match(/[a-zA-Z]/)) {
                    throw new Error('Mật khẩu ít nhất 1 chữ cái và 1 chữ số');
                }
            },
            private: true, //used by the toJSON plugin
        },
        role: {
            type: String,
            enum: roles,
            default: 'user',
        },
    },
    {
        timestamps: true,
    }
);
```

6.2.7. Model của Mongoose

Model là kết quả từ quá trình compile Schema (biên dịch Schema) đã được định nghĩa trước đó. Một instance (thể hiện) của model được gọi là document. Model sẽ chịu trách nhiệm chính trong việc đọc và ghi dữ liệu từ database MongoDB.

Ví dụ tạo Model của accountschema đã tạo ở trên:

```js
//Require gói mongoose
const mongoose = require('mongoose');

//Khai báo kết nối dữ liệu đến cơ sở dữ liệu Quanly
//Kết nối collection: Account
mongoose.connect('mongodb://localhost:27017/Quanly', {
    useNewUrlParrser: true,
    useUnifielTopology: true
});

//Tạo Schema của mongoose
const Schema = mongoose.Schema;

const accountschema = new Schema({
    username: String,
    password: String
}, {
    collection: 'Account'
});

//Tạo model cho accountschema
const AcountModel = mongoose.model('account', accountschema);
```